

Algorithms

It is expected that students are able to develop and interpret algorithms represented as pseudocode and flowcharts.

It is important to start complex algorithms with a clear, uncluttered mainline. The mainline should reference required subroutines, the details of which are shown in separate algorithms.

Each subroutine should be concise and correctly make use of further subroutines for detailed logic.

Pseudocode

Pseudocode is a method of describing the logic in an algorithm. It makes use of capitalised keywords and indentation to show control structures used.

In pseudocode:

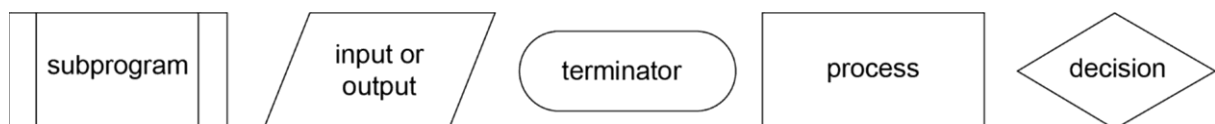
- keywords are written in capitals
- structural elements come in pairs, eg for every BEGIN there is an END, for every IF there is an ENDIF
- indenting is used to identify control structures in the algorithm
- when refining the solution to a problem, a subroutine can be referred to in an algorithm by its name, with a separate subroutine developed with that same name to show the detailed logic.

Flowcharts

Flowcharts are diagrams that represent algorithms and are read from top to bottom and left to right.

Symbols

Flowcharts use the following symbols connected by lines with arrowheads to indicate the flow of data. It is common practice to show arrowheads to avoid ambiguity.



Flowcharts using these symbols should be developed using only the standard control structures (as described in the following section, Control Structures).

Control Structures

Algorithms are developed using the basic control structures of *sequence*, *selection* and *repetition*. Students are expected to design algorithms and write code incorporating combinations of these control structures.

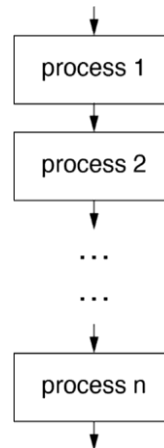
Sequence

Sequence refers to steps which are to be executed one after the other. The steps are executed in the same order in which they are written.

Pseudocode

```
process 1  
  
process 2  
  
...  
  
...  
  
process n
```

Flowchart



Selection

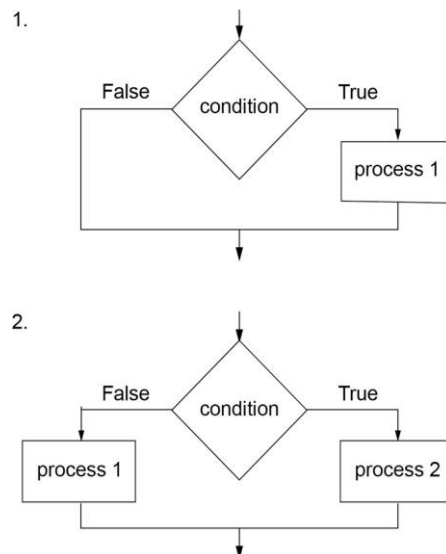
Binary selection

In binary selection, if the condition is met then one path is taken, otherwise the second possible path is followed.

Pseudocode

```
1. IF condition THEN  
    process 1  
ENDIF  
  
2. IF condition THEN  
    process 2  
ELSE  
    process 1  
ENDIF
```

Flowchart



Note: Arrows coming from a decision symbol should be labelled to remove ambiguity.

Multi-way selection

In multi-way selection there can be a number of possible choices, or cases. The path taken is determined by the evaluation of the expression. Once a relevant path has been determined and executed, execution of this expression ceases. Only one process is executed as a result of the implementation of the multi-way selection.

Multi-way selection is often referred to as a *case structure*.

Pseudocode

CASEWHERE expression evaluates to

choice a: process a

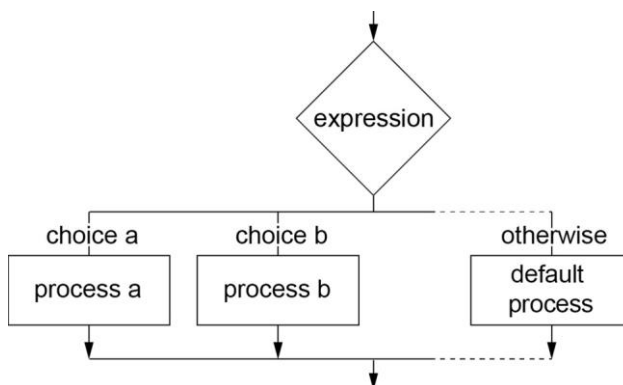
choice b: process b

...

OTHERWISE: default process

END CASE

Flowchart



Nested IF

A nested IF allows the testing of multiple conditions with only one process executed.

Pseudocode

IF condition A THEN

process 1

ELSEIF condition B THEN

process 2

ELSEIF condition C THEN

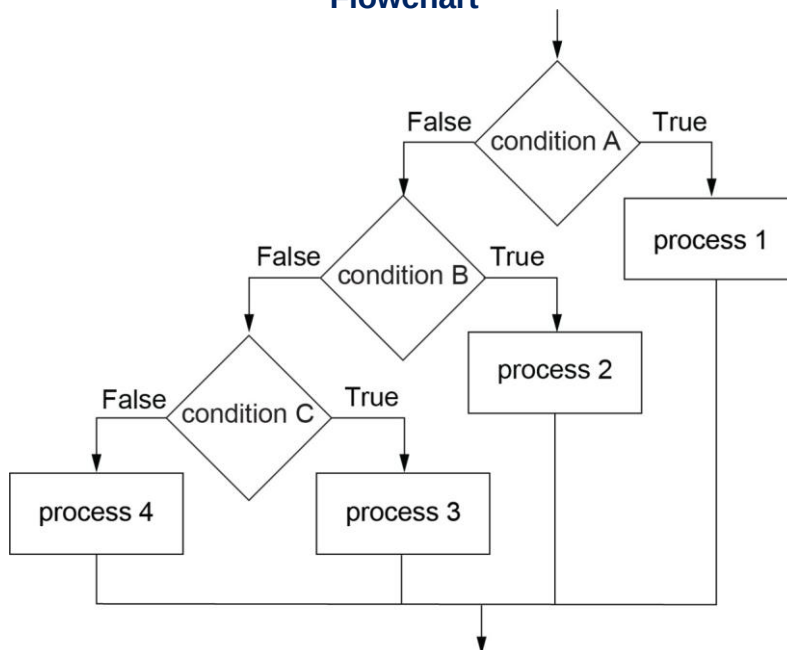
process 3

ELSE

process 4

ENDIF

Flowchart



Repetition

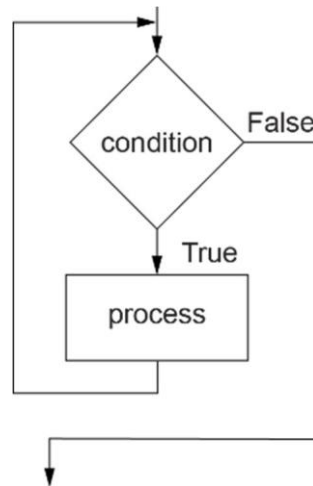
Pre-test

The pre-test loop tests the condition at the start of the loop to determine whether the body of the loop is executed. The body of the loop is executed repeatedly while the termination condition is true.

Pseudocode

```
WHILE condition is true  
    process  
ENDWHILE
```

Flowchart



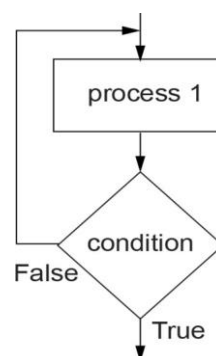
Post-test

A post-test loop executes the body of the loop before testing the termination condition. The body of the loop is repeatedly executed until the termination condition is true.

Pseudocode

```
REPEAT  
    process  
UNTIL condition is true
```

Flowchart



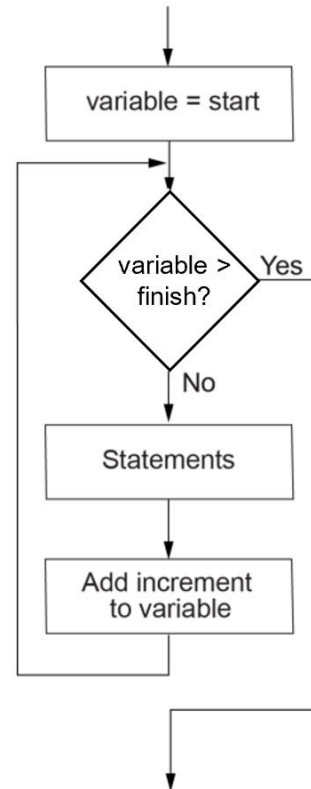
FOR / NEXT

FOR / NEXT loops (also known as counted loops) can be regarded as special cases of repetition and, depending on the language, are implemented as either pre-test or post-test repetitions.

Pseudocode

```
FOR variable = start TO finish STEP increment  
    statements  
NEXT variable
```

Flowchart



Note: Increment can take either a positive or negative value. The flowchart shows the logic for a positive increment

Subroutines

The terms *subroutine*, *module*, *subprogram* and *procedure* can be used interchangeably to represent a collection of statements that achieve a specific purpose.

A subroutine can do the same task at different points in an algorithm. It may operate on different data each time it is called. One or more parameters are used to indicate the data to be processed.

A *function* is a particular type of subroutine that returns a single value.

Using a subroutine with one parameter

The following algorithms represent the logic required to fill an array with characters.

The subroutine *read* uses a single parameter *arrayname* that can take different values.

The first time that this subroutine is called, the data is read and stored in the array called 'name'. The second time, the data is read and stored in the array called 'address'.

Pseudocode

BEGIN

 read (name)

 read (address)

END

BEGIN read (arrayname)

 Set pointer to first position

 Get a character

 WHILE more data AND space in array

 Store data in arrayname at the
 position given by the pointer

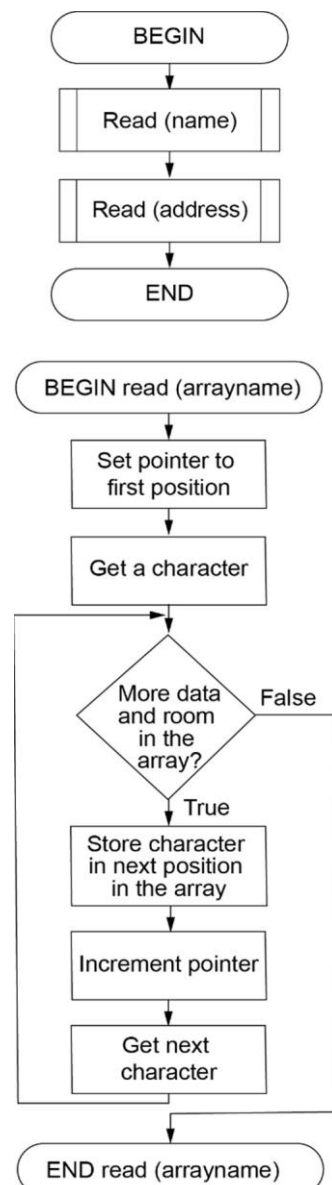
 Increment the pointer

 Get next character

 ENDWHILE

END read (arrayname)

Flowcharts



Using a subroutine with multiple parameters

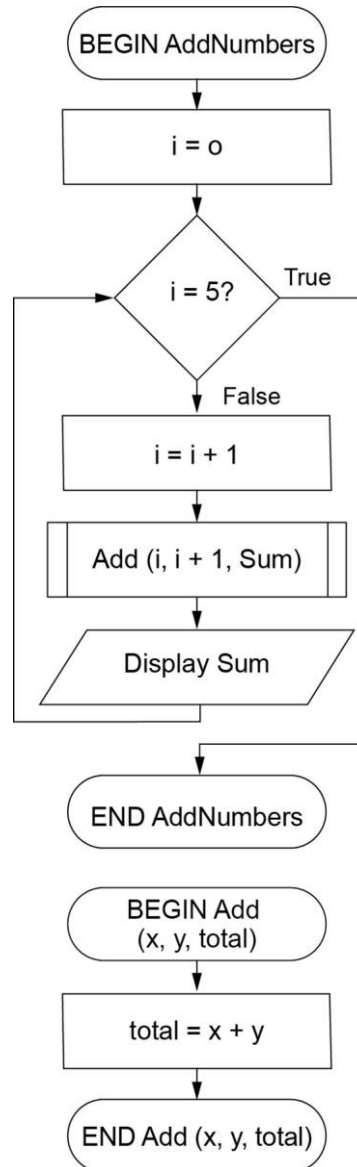
The following algorithms represent the logic required to display the sum of two consecutive integers, where the smaller integer takes all possible values from 1 to 5. The subroutine *Add* uses three parameters, *x*, *y* and *total*.

Pseudocode

```
BEGIN AddNumbers  
    FOR i = 1 to 5  
        Add (i, i + 1, sum)  
        Display sum  
    NEXT i  
END AddNumbers
```

```
BEGIN Add (x, y, total)  
    total = x + y  
END Add (x, y, total)
```

Flowcharts



Passing a value back from a function

A function generates a single value. The word RETURN is used to pass this single value back from the function.

The algorithm *Addnumbers* uses the function *Add* to calculate the sum of two consecutive integers where the smaller integer takes all possible values from 1 to 5. In this case the function requires two parameters.

Pseudocode

```
BEGIN Addnumbers  
    FOR i = 1 to 5  
        Display Add (i, i + 1)  
    NEXT i  
END Addnumbers
```

```
BEGIN Add (x, y)  
    total = x + y  
    RETURN total  
END Add (x, y)
```

Flowcharts

